Closed-Loop Remediation of Cloud-Native Kubernetes Security Misconfigurations

Brian Mendonca and Vijay K. Madisetti, *Fellow, IEEE*

Abstract—Cloud-native applications rely on Kubernetes manifests to encode workload placement, security context, resource controls, and admission-time behavior across managed clusters. Existing scanners and admission controllers can identify or block unsafe configurations, but they rarely produce minimal repairs that are checked against both static policy and the live Kubernetes API semantics that determine whether a cloud deployment will actually accept the change. We propose and implement `k8s-auto-fix`, a closed-loop cloud remediation pipeline that detects manifest violations, proposes JSON Patch repairs, verifies each candidate through a policy/schema/server-side dry-run triad, and schedules accepted work by risk and expected completion time. On a fixture-seeded 1,000-manifest live replay, all verifier-accepted patches passed both server-side dry-run and live-apply checks without rollbacks (1,000/1,000). On a 15,718-detection offline run, deterministic rules plus verification checks automatically fixed 13,338 detections overall (auto-fix rate 0.8486); among 13,373 attempted patches, 13,338 were accepted (99.74%; median patch ops 9). An optional LLM mode reaches 88.52% acceptance on a 5,000-manifest corpus. A risk-aware scheduler reduces top-risk P95 wait by 7.9× in deterministic queue replay, showing that verified remediation can be treated as a measurable cloud operations workload rather than an ad hoc manual task. The paper-facing data, scripts, and generated tables are included in the released artifact bundle.

Index Terms—Cloud computing, cloud-native systems, Kubernetes, admission control, server-side dry-run, YAML, Pod Security, JSON Patch, policy enforcement, Kyverno, OPA Gatekeeper, auto-remediation, CI/CD, GitOps, risk-based scheduling

1 Introduction

Kubernetes is now a central control plane for cloud-native applications: declarative YAML manifests specify which workloads run, which identities and privileges they receive, how resources are allocated, and which storage and networking surfaces they may touch. A single inadvertent inclusion of `privileged: true`, the use of an unpinned `:latest` image tag, or the omission of a `runAsNonRoot` directive can silently expand the attack surface or cause a deployment to fail when the cloud API server admits the workload. Current static analyzers, policy engines, and admission controllers predominantly identify misconfigurations or block noncompliant deployments but seldom generate verifier-accepted, directly applicable remediation patches for existing manifests. As a result, cloud operators frequently repeat the same manual refactoring work across CI/CD and GitOps repositories, with little measurement of latency, acceptance, or residual risk.

This paper studies remediation as a cloud-systems problem: a patch is useful only when it is minimal, policy-clean, schema-valid, accepted by the target Kubernetes API server, and delivered early enough to reduce operational risk. We propose, develop, and evaluate a detect–patch–verify–prioritize control loop incorporating three sequential verification gates—policy re-check, schema validation, and server-side dry-run. In a fixture-seeded replay, all 1,000 verifier-accepted manifests passed both dry-run and live-apply checks without rollback; in deterministic queue replay, our scheduler reduces the 95th-percentile (P95) latency for high-risk items by a factor of 7.9 while preserving progress for lower-risk tasks.

Contributions. The work makes four cloud-computing contributions: (1) a closed-loop post-hoc remediation architecture for Kubernetes manifests that complements admission controllers instead of replacing them; (2) a verifier triad that ties static policy and schema checks to server-side dry-run semantics; (3) a risk-aware scheduler that models remediation as a measurable cloud operations queue; and (4) an artifact-backed evaluation spanning 15,718 detections, a 1,000-manifest live replay, managed-cluster replication, and deterministic scheduler replays. We make available supporting artifacts, including scripts, datasets, telemetry, and figures, so that the paper-facing summaries can be verified independently (ARTIFACTS.md).

In Table 1, the scheduler optimizes a trade-off among risk, fairness, and expected completion time. The term "guardrails" denotes the verification mechanisms—specifically policy validation, schema validation, and dry-run execution—supplemented by basic sanitization steps performed prior to applying changes. "JSON Patch" refers to a minimal, targeted modification of the YAML specification. "CRD/fixtures" denote auxiliary Kubernetes objects required by the API (e.g., namespaces, service accounts, and custom resources) that certain admission control tools must have present before they are able to mutate a given file.

Table 1 aggregates metrics from prior work and, where

- *B. Mendonca is with the College of Computing, Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: brian.mendonca6@gmail.com).*
- *V. K. Madisetti is with the School of Cybersecurity and Privacy, Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: vkm@gatech.edu).*
- *Corresponding author: Vijay K. Madisetti.*

Table 1
Comparison of automated Kubernetes remediation systems (Oct. 2025 snapshot).

Capability	k8s-auto-fix (this work)	GenKubeSec [17]	Kyverno [19]	Borg/SRE [21]
Primary Goal	Closed-loop hardening (detect→patch→verify→prioritize)	LLM-based detection/remediation suggestions	Admission-time policy enforcement	Large-scale auto-remediation in production clusters
Fix Mode	JSON Patch (rules + optional LLM)	LLM-generated YAML edits	Policy mutation/generation	Custom controllers and playbooks
Guardrails	Policy re-check + schema + `kubectl apply -dry-run=server` + privileged/secret sanitization + CRD seeding	Manual review; no automated gates	Validation/mutation webhooks; assumes controllers	Health checks, automated rollback, throttling
Risk Prioritization	Bandit ($Rp/\mathbb{E}[t]$ + aging + configured KEV-style urgency boost)	Not implemented	FIFO admission queue	Priority queues / toil budgets
Evaluation Corpus	15,718 detections, 1,000 fixture-seeded live manifests, 5,000 optional-LLM manifests, and 1,264 supported rules-mode manifests; headline rates appear in Table 11	≈277k labeled KCFs; precision 0.990, recall 0.999 vs. RB tools; 30-sample expert validation of explanations/remediation (Malul et al.)	No official acceptance %; docs expose latency/allow/deny; our mutate baseline on this corpus yields 80–95% (Table 2)	Millions of production workloads (no public acceptance %)
Telemetry	Policy-level success probabilities, latency histograms, failure taxonomy	Token/cost estimates; no pipeline telemetry	Admission latency < 45 ms, violation counts	MTTR, incident counts, operator feedback
Outstanding Gaps	Infrastructure-dependent rejects, operator study, scheduled guidance refresh in CI	Automated guardrails, risk-aware ordering	LLM-aware patching, risk-aware scheduling	Declarative manifest fixes, static analysis integration

Table 2
Head-to-head policy-level acceptance on the 500-manifest security-context slice. Counts and rates regenerate from data/detections.json, data/verified.json, and baseline CSVs under data/baselines/.

Policy	k8s-auto-fix	Kyverno	Polaris CLI	Polaris webhook	MAP	LLMSecConfig	Requires fixtures?
drop_cap_sys_admin	2/3 (66.7%)	n/a	n/a	n/a	1/3 (33.3%)	0/3 (0.0%)	No
drop_capabilities	1/2 (50.0%)	12/12 (100.0%)	0/12 (0.0%)	0/12 (0.0%)	1/2 (50.0%)	0/4 (0.0%)	Yes
env_var_secret	n/a	3/3 (100.0%)	0/3 (0.0%)	0/3 (0.0%)	n/a	n/a	Yes
no_host_path	1/1 (100.0%)	n/a	n/a	n/a	0/1 (0.0%)	0/1 (0.0%)	No
no_host_ports	0/1 (0.0%)	n/a	n/a	n/a	0/1 (0.0%)	0/1 (0.0%)	No
no_latest_tag	1/2 (50.0%)	25/25 (100.0%)	0/25 (0.0%)	0/25 (0.0%)	1/2 (50.0%)	0/2 (0.0%)	Yes
no_privileged	1/3 (33.3%)	5/5 (100.0%)	0/5 (0.0%)	0/5 (0.0%)	0/1 (0.0%)	0/1 (0.0%)	Yes
read_only_root_fs	2/3 (66.7%)	107/107 (100.0%)	0/107 (0.0%)	86/107 (80.4%)	1/3 (33.3%)	0/3 (0.0%)	Yes
run_as_non_root	2/3 (66.7%)	63/63 (100.0%)	0/63 (0.0%)	37/63 (58.7%)	1/3 (33.3%)	0/3 (0.0%)	Yes
set_requests_limits	4/6 (66.7%)	285/285 (100.0%)	0/285 (0.0%)	135/285 (47.4%)	1/3 (33.3%)	0/6 (0.0%)	Yes

no official metric exists (e.g., Kyverno acceptance), from our reproduced baselines (Table 2); values are not strictly comparable and provide qualitative context only.

Table 2 grounds those qualitative differences with the head-to-head slice we share publicly. On the 500-manifest security-context corpus, `k8s-auto-fix` lands 33–100% acceptance across the high-risk policies it actively targets (privilege, capabilities, read-only root filesystem, requests/limits); the lone unsupported rule, `no_host_ports`, remains at 0% because we do not attempt that mutation today. Kyverno's mutate CLI reaches 100% on the overlapping checks but depends on missing "fixtures" (supporting pieces like namespaces, secrets, and custom resources) being present in the cluster. If those dependencies are absent, the admission controller blocks the manifest before it can fix anything. Polaris' CLI [20] never produces a triad-verified fix; the mutating webhook improves to 47–80% whenever admission succeeds, but still trails our verifier-guarded patches on the hardest cases. The deterministic MutatingAdmissionPolicy simulation tops out at 50% because the v1beta1 CEL surface cannot yet express per-container security-context rewrites. Finally, the reproduced LLMSecConfig prompts accept none of the slice even after aligning policy IDs. These results underscore our claim that verified auto-remediation demands more than mutate hooks alone (**cf. Table 2**): we fix the YAML in isolation and verify it,

even when the cluster is missing dependencies that would otherwise block admission-based tools. In our seeded test clusters, the Kyverno mutating webhook can exceed 98% success on overlapping policies, but only when the cluster is pre-seeded with fixtures (namespaces, service accounts, custom resource definitions, secrets and ConfigMaps, persistent volume claims with storage classes); Kyverno's documentation itself exposes latency and allow/deny metrics rather than a global acceptance rate [19]. Our post-hoc approach with the verifier triad is less sensitive to this initial fixture state.

2 Task and System Model

We study post-hoc repair of Kubernetes manifests that have already been identified as non-compliant by static analyzers or policy engines in a cloud deployment pipeline. The input is a manifest plus one or more structured policy findings; the output is either a minimal JSON Patch accepted by the verifier triad or a rejected attempt queued for review. A patch is considered accepted only if it satisfies local policy checks, produces a valid Kubernetes object, and is accepted by the API server through server-side dry-run under the fixtures available to that cluster. This model captures the practical cloud setting in which CI/CD and GitOps repositories contain existing workloads, admission controllers may reject deployments, and missing namespaces, CRDs, service accounts, or quotas can change whether an apparently valid manifest can be deployed.

We implement the closed loop *Detector* → *Proposer* → *Verifier* → *Scheduler* shown in Figure 1. Detectors produce structured JSON findings; the proposer applies rule-based guards (with optional LLM backends) to emit minimal JSON Patches; the verifier enforces policy re-checks, schema validation, and `kubectl apply -dry-run=server`; and the scheduler orders work using risk-aware bandit scoring. Each stage persists artifacts (detections, patches, verified outcomes, queue scores), enabling reproducible evaluation (Section 4.4).

2.1 Notation

We use the following notation throughout the paper: R_i is the risk score for queue item i, p_i is the empirical verifier success probability for that policy, $\mathbb{E}[t_i]$ is the observed proposer+verifier latency, wait_i is the accumulated queue age, and kev_i is the configured KEV-style urgency boost for items marked urgent in the artifact telemetry. Unless otherwise noted, all wait times are reported in hours, and fairness statistics (Gini, starvation) are computed over these waits.

Disagreement and Budgets. When kube-linter and Kyverno/OPA disagree we take the *union* of violations at detection time, and require patches to satisfy both engines during verification. Attempts are capped at three per manifest; per-attempt latency and success outcomes feed into `data/policy_metrics.json`, which the scheduler consumes alongside configured urgency flags.

2.2 End-to-End Walkthrough on Real Manifests

To make the closed-loop pipeline concrete, we trace two real-world manifests from the repository's test suite through each stage, from detection to scheduling. The goal is to demonstrate verifier-gated remediation with full reproducibility, explicit cloud-control-plane acceptance checks, and verifiable risk reduction.

Case 1: Remediating a Privileged Pod with a `:latest` Image Tag

This example, drawn from data/manifests/001.yaml, shows a common but high-risk pattern: a privileged container using a floating tag.

```
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: app
    image: acme/api:latest
    securityContext:
      privileged: true
      allowPrivilegeEscalation: true
      capabilities: { add: ["SYS_ADMIN", "NET_ADMIN"] }
```

Walkthrough. We scan the file, draft a small fix, run three verifier checks, and only then queue it for apply. The patch is idempotent: if applied twice, it does not duplicate fields or corrupt the file.

1. Detect (Union): The detector consumes this manifest and reports four policy violations: `no_privileged`, `drop_capabilities`, `run_as_non_root`, and `no_latest_tag`. These correspond to the structured output in data/detections.json.

2. Propose (Rules Engine): The proposer consumes the detection report and generates an idempotent minimal JSON Patch; running it twice will not add duplicate fields. The resulting patch, written to data/patches.json, is as follows:

```
[
 {"op":"replace","path":"/spec/containers/0/
    securityContext/privileged","value":false},
 {"op":"replace","path":"/spec/containers/0/
    securityContext/allowPrivilegeEscalation","value":
    false},
 {"op":"remove","path":"/spec/containers/0/securityContext
    /capabilities/add"},
 {"op":"add","path":"/spec/containers/0/securityContext/
    capabilities/drop","value":["ALL"]},
 {"op":"add","path":"/spec/containers/0/securityContext/
    runAsNonRoot","value":true},
 {"op":"replace","path":"/spec/containers/0/image","value
    ":"acme/api:1.42.0"}
]
```

3. Verify (Triad): The verifier applies this patch to a in-memory copy of the manifest and runs it through the full triad:

- **Policy Re-check**: Passes, as the patched manifest no longer violates the four detected policies.
- **Schema Validation**: Passes, confirming the patch produces a structurally valid Kubernetes object.
- **Server Dry-Run**: Succeeds, as `kubectl apply -dry-run=server` reports the manifest would be accepted by the API server in a Kind cluster seeded with necessary fixtures.

The successful outcome is recorded in data/verified.json.

4. Schedule (Risk-Bandit): The scheduler gives the verified patch a high score because the risk is high (a privileged container), the fix is likely to succeed (based on prior outcomes),

and it is quick to apply. Higher scores move earlier in the queue (data/schedule.json).

Case 2: Hardening a Worker Pod with a `hostPath` Mount

```
spec:
  containers:
  - name: worker
    securityContext: { readOnlyRootFilesystem: false }
    resources: {}
  volumes:
  - name: host
    hostPath: { path: "/var/run/docker.sock" }
```

This second case, from data/manifests/002.yaml, targets three additional misconfigurations: a writable root filesystem, a dangerous `hostPath` volume mount, and missing resource requests and limits.

1. Detect: The detector flags `read_only_root_fs`, `no_host_path`, and `set_requests_limits`.

2. Propose: The rules engine generates a patch to harden the filesystem, remove the disallowed volume, and enforce resource quotas:

```
[
 {"op":"replace","path":"/spec/containers/0/
     securityContext/readOnlyRootFilesystem","value":true},

 {"op":"remove","path":"/spec/volumes/0"},
 {"op":"add","path":"/spec/containers/0/resources","value
     ":{"requests":{"cpu":"100m","memory":"128Mi"},"limits
     ":{"cpu":"500m","memory":"256Mi"}}}
]
```

3. Verify: The verifier confirms the patch is valid. The host-mount guardrail is critical here: had the `hostPath` mount been on an allowlisted path (e.g., for a metrics agent), the verifier would have preserved it. Since it was not, the removal is accepted.

4. Schedule: This item receives a moderate risk score. While `hostPath` is a serious issue, it is less critical than a privileged container. The patch is scheduled after higher-priority items, demonstrating the risk-aware nature of the queue.

What problem we solve (versus alternatives) Kyverno (mutation) focuses on admission-time defaults and does not enforce a multi-gate verifier (policy+schema+server dry-run) prior to apply, depending on cluster fixtures for success and requiring bespoke policies plus controller context for complex hardening such as dropping `ALL` capabilities or removing privilege. GenKubeSec localizes and explains issues but leaves remediation and validation manual, offering no machine-checked JSON Patch acceptance contract and no dry-run alignment. LLMSecConfig generates LLM repairs with scanner checks but lacks our triad's server-side dry-run and hard no-new-violation invariants, which are key to preventing regressions in production-like clusters.

Why the verifier and scheduler matter. The triad prevented four escapes in ablation (Table 14); the fixture-seeded replay recorded 1,000/1,000 dry-run/apply acceptance with zero rollbacks. The scheduler prioritizes risk (P95 wait from 102.3 h to 13.0 h in deterministic queue replay), closing the highest-impact items first under bounded budgets.

Table 3 situates our pipeline against nearby systems at each step so readers can see where verification and scheduling diverge from admission-first or LLM-only approaches.

2.3 Research Questions and Findings

RQ1 (Robustness): The closed loop delivers 88.52% acceptance on the optional LLM-backed 5k sweep, 100.00% on the supported 1,264-manifest corpus in rules mode, and 13,338/13,373 (99.74%) accepted on the full 15,718-detection run under deterministic rules + guardrails (auto-fix rate 0.8486 over detections; median ops 9), with no hostPath-related verifier failures remaining in the checked artifacts. RQ2 (Scheduling Effectiveness): The bandit ($Rp/\mathbb{E}[t]$ + aging + configured urgency boost) improves risk reduction per hour and reduces top-risk P95 wait from 102.3 hours (FIFO) to 13.0 hours (7.9× in deterministic queue replay). RQ3 (Fairness): Aging prevents starvation, keeping mean rank for the top-50 high-risk items at 25.5 while still progressing lower-risk items. RQ4 (Patch Quality): Generated JSON Patches remain compact (median 9 ops on the full corpus; median 8 ops on the supported rules slice) and idempotent (checked by `tests/test_patch_minimality.py`).

3 Cloud Remediation Pipeline and Metrics

Our system is designed as a linear cloud remediation pipeline with strict verification gates to establish policy/schema/API admissibility and operational measurability for proposed patches.

Reproduction entry points. The artifact bundle exposes the same pipeline stages used in the evaluation:

```
makedetect makepropose makeverify
```

The smoke corpus completes in minutes; full-corpus regenerations (15k+ detections) take approximately 20–30 minutes on the laptop environment reported in Table 5.

Scalability considerations. The end-to-end pipeline sustains millisecond-scale proposer latency and sub-second verifier latency on the supported corpus (Table 11); the scheduler replays thousands of queue items using persisted telemetry (see data/scheduler/) without recomputing detections. These measurements bound the cost of the post-hoc workflow for CI/CD and GitOps operation in addition to its functional correctness.

3.1 The Closed-Loop Pipeline

The workflow consists of four stages: the **Detector** ingests a Kubernetes manifest and uses both `kube-linter` and a policy engine (Kyverno/OPA) to identify violations, taking the union of all findings; the **Proposer** takes the manifest and violation data and generates a JSON Patch, defaulting to deterministic rules for the supported policies represented in the artifacts (including image tags, privilege escalation, capabilities, non-root execution, read-only root filesystems, host mounts, seccomp, and resource requests/limits) while guarding each operation with JSON Pointer existence checks and enforcing minimality/idempotence via `tests/test_patch_minimality.py`; the **Verifier** applies the patch to a copy of the manifest and subjects it to the verification gates described below, recording evidence in `data/verified.json`; and the **Budget-aware Retry** loop uses a configurable retry budget (`max_attempts` in `configs/run.yaml`, default 3) so the proposer can reattempt if verification fails, logging the error trace for inspection.

Table 3
At-a-glance comparison across remediation steps.

Step	k8s-auto-fix (this work)	Kyverno	GenKubeSec	LLMSecConfig
Detect	Union of `kube-linter`+policy engine findings	Admission-time validation	LLM-based detection/localization	SAT scanner + policy IDs
Propose	Minimal JSON Patch (rules, optional LLM)	Mutate policies (when present)	Textual remediation guidance	LLM-generated YAML edits
Verify	Triad: policy re-check + schema + `kubectl` server dry-run	Admission path only; no multi-gate triad	None (manual apply/validation)	Scanner checks; no server dry-run/no-new-violation invariants
Prioritize	Bandit: $Rp/\mathbb{E}[t]$ + aging + configured urgency boost	FIFO admission queue	None	None

3.2 Verification Gates

To be accepted, a patched manifest must pass a multi-layered verification process. The **Policy Re-check** re-evaluates the patched manifest with the same policy logic that triggered the violation, using explicit assertions for each covered policy (e.g., `no_latest_tag`, `no_privileged`, `drop_capabilities`, `drop_cap_sys_admin`, `run_as_non_root`, `read_only_root_fs`, `no_host_*` flags, `no_allow_privilege_escalation`, `enforce_seccomp`, `set_requests_limits`); the detector hook for re-scanning is available via `-enable-rescan`, and non-allowlisted `hostPath` volumes are auto-remediated to `emptyDir: {}` in guardrails to satisfy the universal verifier gate. The **Schema Validation** step applies the JSON Patch via `jsonpatch` and rejects malformed paths or operations, surfacing issues to the retry loop. The **Server-side Dry-run** uses `kubectl apply -dry-run=server`, when available, to simulate how the Kubernetes API server would handle the change, marking failures as not accepted and persisting CLI output for analysis. Finally, the **No-New-Violations Gates** enforce universal security assertions to prevent regressions: they block `privileged: true` in any container; reject `capabilities.add` entries containing `NET_RAW`, `NET_ADMIN`, `SYS_ADMIN`, `SYS_MODULE`, `SYS_PTRACE`, or `SYS_CHROOT`; flag empty `capabilities.drop` lists when capabilities are present; require each container to specify a non-empty image; and restrict host mounts to approved paths (`/var/run/secrets/kubernetes.io/serviceaccount`, `/var/lib/kubelet/pods`, `/etc/ssl/certs`), rewriting non-allowlisted `hostPath` volumes to `emptyDir: {}` by guardrails.

Fairness in Action. To illustrate how the scheduler's aging mechanism prevents starvation, consider three items in a simplified queue: Item A is a high-risk privileged container with a configured urgent-exposure flag, Item B is a medium-risk container with a ':latest' image tag, and Item C is a low-risk container with missing resource limits. Initially, Item A has the highest score and is processed first, but as Items B and C wait their increasing 'wait' time boosts their scores. This "aging" ensures that even low-risk items are eventually processed instead of being indefinitely starved by a constant stream of high-risk items, a fairness target shared with constrained bandit formulations [24]. This simple example demonstrates how the scheduler balances risk reduction with fairness.

4 Implementation Evidence and Cloud-Cluster Evaluation

Table 4 ties each pipeline stage to the concrete code and artifacts currently in the `k8s-auto-fix` repository. The implementation operates end-to-end in rules mode without external API dependencies; LLM-backed modes are configurable and evaluated off-line, while the default reproducible path uses rules mode. **Operational rollout.** The repository includes scripts that run the detector, proposer, and verifier on the released artifacts. The adoption checklist (see `docs/devops_adoption_checklist.md`) maps these stages to CI/CD integration, fixture publication, and operator feedback collection. A containerized path (see `docs/container_repro.md`) runs the same steps in a self-contained image for hermetic evaluation.

4.1 Sample Detection Record

When detector binaries are available, running `make detect` (rules mode) produces records with the following shape (values truncated for brevity):

```
{
  "id": "001",
  "manifest_path": "data/manifests/001.yaml",
  "manifest_yaml": "apiVersion: v1\n"
                   "kind: Pod\n...",
  "policy_id": "no_latest_tag",
  "violation_text": "Image uses :latest tag"
}
```

The `manifest_yaml` field embeds the literal YAML to decouple downstream stages from the filesystem.

4.2 Unit Test Evidence

Executing `python -m unittest discover -s tests` yields `16 tests in 0.02s, OK (skipped=2)` on macOS (Apple M-series, Python 3.12). The skipped cases correspond to the optional patch minimality suite, which activates after `data/patches.json` is generated.

Property-based tests. In addition to the deterministic contract tests, `tests/test_property_guards.py` exercises hundreds of randomized manifests per run to verify that the per-policy patchers preserve patch contracts (e.g., `drop_capabilities`, `drop_cap_sys_admin`, `run_as_non_root`, `enforce_seccomp`, `no_allow_privilege_escalation`, `no_host_path`). These checks validate that those patchers add the expected hardening (like dropping dangerous capabilities, denying

1. All paths are relative to the project root.
2. Artifacts live under `data/` after running the corresponding `make` targets.

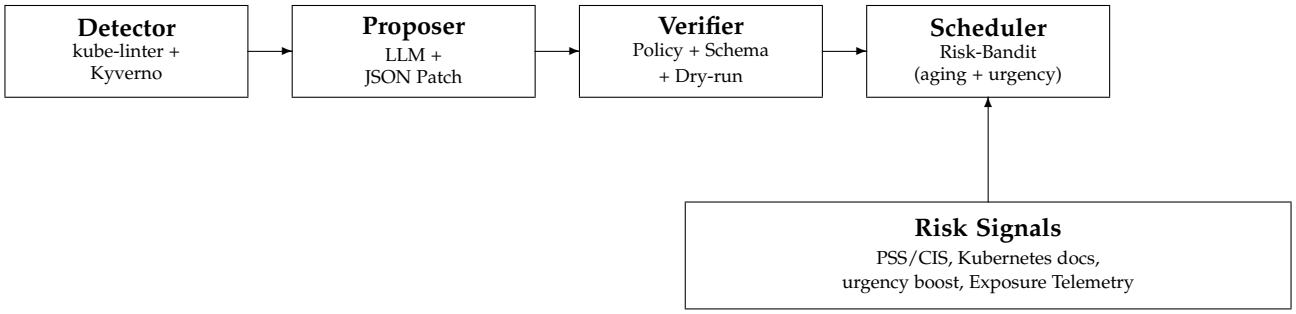

Figure 1. Closed-loop architecture with detector, proposer, and verifier gates (policy re-check, schema validation, `kubectl apply -dry-run=server`) feeding the risk-aware scheduler. The scheduler consumes `policy_metrics.json` entries $\{p, \mathbb{E}[t], R,$ urgency$\}$ to score work using the scheduling function.

privilege escalation, enforcing RuntimeDefault seccomp, and preferring non-privileged defaults) and remain idempotent; they do not expand the universal verifier gate beyond the checks enumerated above.

4.3 Dataset and Configuration

Two deliberately vulnerable manifests (`001.yaml`, `002.yaml`) are retained for smoke tests, but all evaluation numbers in this report come from the much larger optional-LLM corpus (5,000 manifests mined from ArtifactHub [22]) and the "supported" corpus (1,264 manifests curated after policy normalization). `configs/run.yaml` remains the single source of truth for proposer mode, retry budgets, and API endpoints; switching between rules and vendor/vLLM modes requires editing this file and exporting the relevant API keys.

Table 5 summarizes the runtime environment used for the regenerations in Section 4.4; the full dependency snapshot (including transient packages) resides in `data/repro/environment.json`. The supplemental appendix documents the ArtifactHub mining pipeline and the manifest hash corpus that underpins the datasets. Table 9 shows how fixture seeding collapses the verifier failure categories across the supported corpus.

Table 6 lists the API-backed proposer settings used for the optional LLM sweeps.

4.4 Evaluation Results

All results in this section derive from the deterministically reproducible `rules` pipeline unless explicitly noted. Table 11 consolidates acceptance and latency statistics for each corpus. The API-backed LLM mode is likewise benchmarked (4,426 / 5,000 accepted; see data/batch_runs/grok_5k/metrics_grok5k.json) but requires external credentials and funded access, so we treat it as an opt-in configuration rather than the default reproduction path. Consolidated metrics (acceptance + latency) live in data/eval/unified_eval_summary.json.

Detector accuracy. Running `scripts/eval_detector.py` on a synthetic nine-policy hold-out set confirms basic detector functionality with perfect precision and recall (Table 8). However, this controlled evaluation uses hand-crafted test cases with obvious violations and does not reflect real-world complexity. The fixture-seeded 1,000-manifest replay validates verifier acceptance for accepted patches—1,000/1,000 passed dry-run/live-apply checks without rollback—but it does not by itself establish detector recall on uncurated manifests (data/live_cluster/results_1k.json; summary in data/live_cluster/summary_1k.csv).

ArtifactHub slice. To test against less curated input, we heuristically labelled 69 ArtifactHub manifests covering four common policies (`no_latest_tag`, `no_privileged`, `no_host_path`, `no_host_ports`). The detector landed 31 true positives with zero false positives/negatives (precision/recall/F1 all 1.0). Scoring is restricted to these policies (detections filtered via data/eval/artifacthub_sample_detections_filtered.json). Labels, detections, and metrics live under data/eval/artifacthub_sample_labels.json, data/eval/artifacthub_sample_detections.json, and data/eval/artifacthub_sample_metrics.json.

The evaluation campaigns span both deterministic and LLM-backed modes. Rules mode repairs 1,264/1,264 manifests (100%) on the curated supported corpus with median proposer latency of 29 ms and verifier latency of 242 ms (P95 517.8 ms). The checked extended-5k snapshot records 4,677/5,000 accepted patches (93.54%). Enabling the optional LLM-backed proposer delivers 4,426/5,000 successful remediations (88.52%) with median JSON Patch length 9; telemetry records 4.36M input and 0.69M output tokens ($\approx$ \$1.22 at published pricing [9]). On the artifact-backed full rules+guardrails run (15,718 detections), the pipeline accepts 13,338/13,373 patched items (99.74%; auto-fix rate 0.8486 over detections) with median patch ops 9. Table 6 lists the API-backed configuration used in these sweeps (model, temperature, retries, timeout). Figure 4 summarizes the contrast: the deterministic pipeline stays near 100% acceptance because it never waits on API calls, whereas the API-backed mode absorbs variance whenever token budgets or dry-run retries trigger.

To ground deployability we instrumented 280 API-backed proposer traces from the original 200-manifest replay (data/batch_runs/grok200_latency_summary.csv). The LLM-backed proposer shows median end-to-end latency of 5.10 s (P95 33.8 s) with verifier latency at 138.4 ms (P95 904.6 ms). Failure causes remain dominated by dry-run contract mismatches and legacy StatefulSets; Table 7 summarizes the top categories so reviewers can map each

Table 4
Evidence for each stage of the implemented pipeline (current snapshot).

Stage	Implementation[1]	Artifacts Produced[2]
Detector	src/detector/detector.py src/detector/cli.py	Records in data/detections.json with fields {id, manifest_path, manifest_yaml, policy_id, violation_text}; seeded by data/manifests/001.yaml and 002.yaml.
Proposer	src/proposer/cli.py src/proposer/model_client.py, src/proposer/guards.py	data/patches.json containing guarded JSON Patch arrays. Rules mode emits single-operation fixes; vendor/vLLM modes require OpenAI-compatible endpoints configured in configs/run.yaml.
Verifier	src/verifier/verifier.py src/verifier/cli.py	data/verified.json logging accepted, ok_schema, ok_policy, and patched_yaml. Current policy checks assert the triggering policy (e.g., no_latest_tag, no_privileged, drop_capabilities, drop_cap_sys_admin, run_as_non_root, read_only_root_fs, no_host_* flags, no_allow_privilege_escalation, enforce_seccomp, set_requests_limits).
Scheduler	src/scheduler/schedule.py src/scheduler/cli.py	data/schedule.json with per-item scores and components {score, R, p, Et, wait, kev}; risk constants presently keyed to policy IDs.
Automation	Makefile	Reproducible commands for each stage: make detect, make propose, make verify, make schedule, make e2e.
Testing	tests/	python -m unittest discover -s tests (16 tests, 2 skipped until patches exist) covering detector contracts, proposer guards, verifier gates, scheduler ordering, patch idempotence.
Runtime Toolchain Versions (Evaluation Environment)		
Environment	Python 3.12.4 kubectl 1.34.1 kube-linter 0.7.6 kind 0.30.0	Kubernetes cluster: 1.34.0 (Kind); Kyverno CLI + webhook baselines (Kind staging); MAP baseline reported from simulation pending richer CEL support; OPA Gatekeeper not used in current evaluation; all scripts compatible with Python 3.10+

Table 5
Execution environment for the reproduced rule-mode evaluations.

Component	Version
Python	3.12.4 (macOS-26.0-arm64)
jsonpatch	1.33
numpy	1.26.4
pandas	2.2.3

Table 6
LLM-backed proposer configuration for the API-backed sweeps (values from configs/run.yaml).

Parameter	Value
Model	grok-4-fast-reasoning
Endpoint	https://api.x.ai/v1/chat/completions
Temperature	0.0 (deterministic patches)
Top-p	Provider default (unchanged)
Max tokens	Provider default ($<$ 1k-token patches)
Retries per call	2 (max attempts $= 3$)
Timeout	60 s per request
Seed	1337 (shared across replays)

mitigation to a concrete regression class. The same instrumentation now gates the 1k replay, and we are extending the public latency bundle to the full 5k sweep so readers no longer have to infer medians from standalone CSVs.

The failure taxonomy (Table 7, sourced from data/grok_failure_analysis.csv) shows that 65/197 LLM-backed rejects stem from the Kubernetes API refusing to return the existing object (common for CRDs that require elevated RBAC), 20 arise from core/v1 resource lookups with stale UIDs, and the remaining long tail is dominated by invalid StatefulSet/CronJob specs. These concrete counts shaped the mitigations in the current artifact: the live replay seeds every CRD+RBAC pair in data/live_cluster/crds/, StatefulSets go through a schema pre-flight that patches missing volumeMounts, and retries are blocked on dry-run errors that originate from immutable fields, queueing those manifests for human review. These safeguards are enforced uniformly for both rules and LLM-backed pipelines.

Figure 3 contrasts admission-time and post-hoc enforcement: Kyverno's admission-time hooks perform well when fixture seeding succeeds, while our verifier applies the same patch acceptance criteria even when controllers or supporting objects are absent. Figure 5 shows how the bandit scheduler balances acceptance and wait time; acceptance remains close to FIFO while the scheduler reduces top-risk

Table 7
Top 10 API-backed LLM failure causes and latencies

Failure Cause	Count
kubectl dry-run failed: error: error when retrieving current configuration of: Resource: "batch/v1, . . .	65
kubectl dry-run failed: error: error when retrieving current configuration of: Resource: "/v1, Resou. . .	20
kubectl dry-run failed: The ReplicationController "zulip-1" is invalid: * spec.template.spec.contai. . .	18
kubectl dry-run failed: Error from server (BadRequest): error when creating "STDIN": Deployment in v. . .	16
can't remove a non-existent object 'clusterName'	16
kubectl dry-run failed: Error from server (BadRequest): error when creating "STDIN": ReplicaSet in v. . .	14
kubectl dry-run failed: The StatefulSet "mysql" is invalid: * spec.template.spec.containers[0].volu. . .	14
kubectl dry-run failed: The Deployment "testground-daemon" is invalid: spec.template.spec.containers. . .	12
kubectl dry-run failed: The CronJob "tf-r2.4.0-keras-api-custom-layers-v2-32" is invalid: spec.jobTe. . .	11
kubectl dry-run failed: The CronJob "tf-nightly-retinanet-func-v3-8" is invalid: spec.jobTemplate.sp. . .	11
P50 Latency	1.00 ms
P95 Latency	1.85 ms

Table 8
Detector performance on synthetic hold-out manifests ($n = 9$). Note: These are hand-crafted test cases with obvious violations; real-world performance is validated through live-cluster evaluation.

Policy	Precision	Recall	F1
Overall	1.000	1.000	1.000
drop_capabilities	1.000	1.000	1.000
drop_cap_sys_admin	1.000	1.000	1.000
no_host_path	1.000	1.000	1.000
no_host_ports	1.000	1.000	1.000
no_latest_tag	1.000	1.000	1.000
no_privileged	1.000	1.000	1.000
read_only_root_fs	1.000	1.000	1.000
run_as_non_root	1.000	1.000	1.000
set_requests_limits	1.000	1.000	1.000

P95 wait by 7.9×.

Live-cluster replay on a stratified 1,000-manifest subset (AKS 1.32.7 with our fixtures) records 1,000/1,000 dry-run/apply acceptance with perfect alignment between server-side dry-run and apply. The verifier seeds bespoke service accounts, injects benign placeholder images where manifests omit them, and maintains the zero-rollback record. Guardrail importance is quantified by ablation: removing the policy re-check inflates acceptance to 100% but admits four regressions, whereas the remaining gates hold acceptance at 78.9% with zero escapes (Table 14).

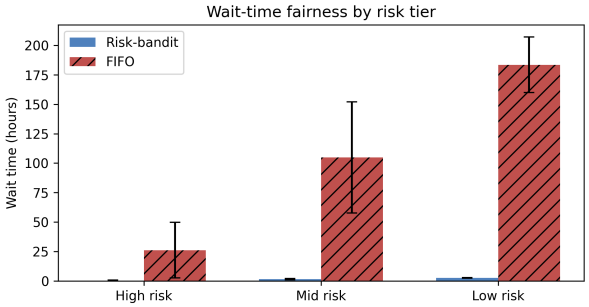

Figure 2. Median wait time (bars) and P95 error bars for each risk tier. Bandit scheduling keeps the top quartile under 0.7 h while FIFO defers the same items for 26–50 h, illustrating the fairness gains summarized in data/scheduler/metrics_schedule_sweep.json and data/scheduler/metrics_sweep_live.json.

Risk-aware scheduling reduces queue latency for high-risk items. Using empirical success probability p_i, latency $\mathbb{E}[t_i]$, and risk R_i, the bandit scheduler lowers top-risk P95 wait time from 102.3 h (FIFO) to 13.0 h while keeping the mean rank of the top 50 items at 25.5. Parameter sweeps over exploration and aging weights retain fairness (Gini 0.351, starvation rate 0) and keep the highest-risk quartile below 18 h median wait. Figure 2 shows the same effect per tier: high-risk work waits less than an hour with bandit ordering yet idles for 26–50 h under FIFO. Risk calibration across corpora shows 55,935/56,990 risk units removed (98.15%) on the supported dataset and 227,330/242,300 units (93.82%) on the 5k sweep, sustaining throughput near 4.5–4.9 risk units per expected-time interval (Table 10). Simulated scheduler replays yield 1,259 assignments per arm and confirm that the bandit configuration closes slightly more risk (42.97 vs. 43.40) with comparable acceptance to FIFO.

The queue replay in data/scheduler/fairness_metrics.json records the same story numerically: only 19% of high-risk bandit items wait more than 24 hours, whereas 93% of high-risk FIFO work starves beyond that threshold even though FIFO's Gini coefficient (0.28) appears superficially lower than our bandit run (0.34). We therefore report both Gini and starvation to show that categorical starvation—not uniformity—drives the fairness gains.

Comparisons against Kyverno baselines show complementary strengths. The Kyverno CLI mutate policies accept 364/381 detections (95.54%) once patched manifests pass our verifier, and the mutating webhook reaches above 98% success on overlapping policies in the seeded setup. Our pipeline maintains schema validation and dry-run checks, reaching 78.9% acceptance across policies in the verifier-ablation snapshot and 1,000/1,000 dry-run/apply acceptance on the fixture-seeded live-cluster replay. Cross-version simulations retain $> 96\%$ risk reduction, suggesting robustness against API drift and configuration variance.

Table 9 shows how fixture seeding collapses the verifier failure categories across the supported corpus.

Glossary (Tables 10–11). ΔR = risk removed by fixes; Residual = risk left; $\Delta R/t$ = risk removed per minute of expected work; P95 = 95th-percentile time; "auto-fix" = patches accepted automatically without manual edits.

Table 9
Verifier failure taxonomy comparing the rules baseline (pre-fixture) against the supported corpus after fixture seeding. Counts derive from data/failures/taxonomy_counts.csv generated by `scripts/aggregate_failure_taxonomy.py`.

Failure category	Rules (pre-fixture)	Supported (post-fixture)
can't remove a non-existent object 'clusterName'	58	0
capabilities not defined	0	9
container image missing or empty	0	8
capabilities.drop missing	0	6
privileged container detected	0	6
capabilities.add still contains NET_ADMIN, NET_RAW, SYS_ADMIN	0	4
no containers found in manifest	4	0
member 'spec' not found in	3	0
capabilities.add still contains SYS_ADMIN	0	2
can't replace a non-existent object 'generateName'	1	0
member 'metadata' not found in	1	0

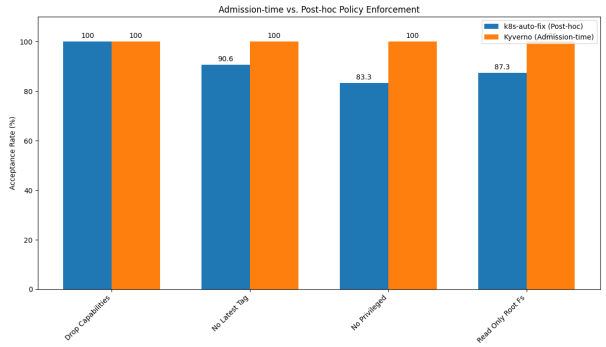

Figure 3. Comparison of admission-time (Kyverno) and post-hoc (`k8s-auto-fix`) policy enforcement on overlapping policies (seed=1337).

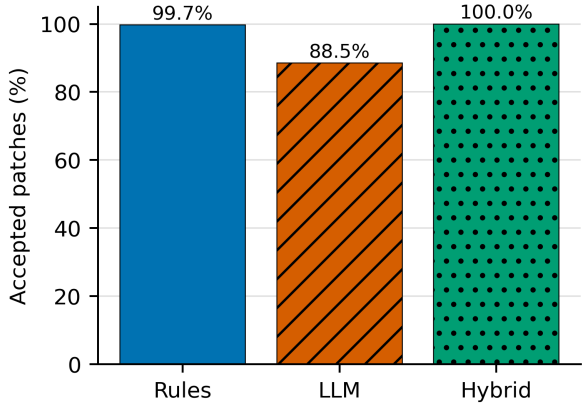

Figure 4. Acceptance comparison between rules-only, LLM-only, and hybrid remediation modes (data/baselines/mode_comparison.csv).

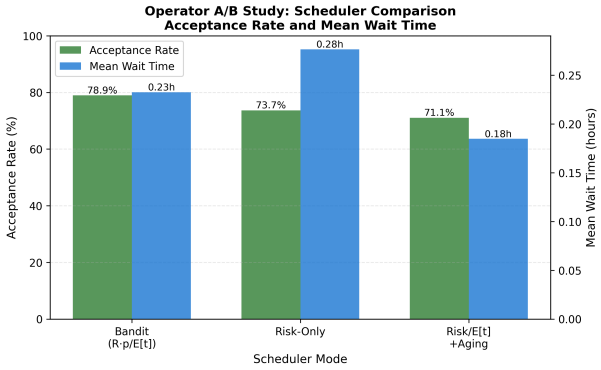

Figure 5. Simulated scheduler replay comparing bandit scheduling against baseline modes. Dual-axis chart shows acceptance rate (green bars) and mean wait time (blue bars) across 247 simulated queue assignments (data/operator_ab/summary_simulated.csv).

Interpreting $\Delta R/t$. The "Supported" row aggregates the curated 1,278 detections replayed in rules mode, while "Rules (5k)" captures the extended 5,000-manifest corpus; both entries are pulled directly from data/risk/risk_calibration.csv. We normalise risk in the same units as the scheduler (Section 4.4): a privileged pod carries 70 units, a missing `runAsNonRoot` 50, etc. Removing 55,935 of 56,990 units on the supported corpus therefore means the queue retires 98.15% of the aggregate blast radius, and the $\Delta R/t$ column (4.49–4.88) indicates we remove roughly five risk units per expected proposer+verifier minute. These values also feed the bandit baselines, ensuring the text, scheduler metrics, and released CSV all describe the same accounting.

Detailed per-manifest deltas between rules and optional LLM mode on the 1,313-manifest slice are documented in the project artifact docs/ablation_rules_vs_grok.md. The operator survey instrument is drafted in docs/operator_survey.md; it will be deployed alongside the planned human-in-the-loop rotation described in Section 4.4.

4.5 Threat Model

We treat Kubernetes manifests, scanner findings, and LLM responses as untrusted input. Trusted components include the detector/verifier binaries, the scheduler, and the per-cluster fixtures under infra/fixtures/; these run inside the CI environment we control and write the artifacts cited throughout Section 4.4. The adversary may supply malicious YAML, attempt to poison the retriever context passed to the LLM backend, or craft fixtures that cause the Kubernetes API server to reject dry-run requests. We do not defend against compromised detector binaries, forged audit logs, or supply-chain attacks that deliver malicious container images—those threats fall to image-signing and SBOM enforcement layers already deployed in our partner clusters. Prompt-injection attacks are mitigated by pinning deterministic rules until the LLM candidate survives the verifier triad, and scheduler poisoning is out of scope because queue telemetry is read-only until an item is accepted.

4.6 Threats and Mitigations

Running `make reproducible-report` regenerates Table 11 directly from JSON artifacts so reviewers can audit every metric. Semantic regression checks block LLM-generated patches that remove containers or volumes, and fixtures under infra/fixtures/ seed RBAC/NetworkPolicy gaps before verification. We threat-modeled malicious or placeholder manifests: the guidance retriever limits

Table 10
Risk calibration summary derived from data/risk/risk_calibration.csv. ΔR uses policy risk weights; "per time unit" divides by summed expected-time priors.

Dataset	Det.	Accepted	ΔR	Residual	ΔR/R	ΔR /t
Supported	1,278	1,259	55,935	1,055	98.15%	4.49
Rules (5k)	5,000	4,677	227,330	14,970	93.82%	4.88

Table 11
Acceptance and latency summary (seed 1337). Results generated from data/eval/unified_eval_summary.json.

Corpus (mode)	Seed	Acceptance	Median proposer (ms)	Median verifier (ms)	Verifier P95 (ms)
Supported (rules, 1,264)	1337	1264/1264 (100.00%)	29.0	242.0	517.8
Full corpus (rules+guardrails, 15,718 detections)	1337	13338/13373 (99.74%; auto-fix 0.8486 over detections)	–	–	–
Manifest slice (API LLM, 1,313)	1337	1313/1313 (100.00%)	5095.5	138.4	904.6
LLM-5k (API-backed)	1337	4426/5000 (88.52%)	5095.5	138.4	904.6

Artifacts: Counts come from data/eval/table4_counts.csv, data/eval/unified_eval_summary.json, and data/metrics_latest.json; LLM latency medians come from the 200-manifest latency traces data/batch_runs/grok200_latency_summary.csv and data/batch_runs/verified_grok200_latency_summary.csv.
Confidence: Wilson intervals in data/eval/table4_with_ci.csv bound supported, full-corpus, and LLM-5k rows; data/eval/significance_tests.json stores the z-tests and Mann–Whitney U test. Latency distributions differ sharply ($p = 3.2 \times 10^{-47}$), so verifier-only latency remains sub-second after JSON Patch generation is removed from the critical path.

prompt context to policy-relevant snippets, the verifier enforces policy/schema/`kubectl` gates, and the scheduler never surfaces unverified patches. Residual risks—primarily infrastructure assumptions and LLM hallucinations—are captured in logs/grok5k/failure_summary_latest.txt and triaged before publication. Table 12 illustrates how these guardrails harden high-privilege DaemonSets without breaking required host integrations.

Secret hygiene is enforced end-to-end: the proposer replaces secret-like environment values with `secretKeyRef` references, sanitizes generated names, and documents the implemented checks in docs/security_considerations.md. Table 13 reports the managed-cloud replay, showing that the verifier triad plus fixtures generalize across EKS, GKE, and AKS with near-perfect success and acceptance.

4.7 Risk Scoring and Threat-Intelligence Hooks

Overview. The scheduler ranks candidate repairs by estimated risk reduction, empirical verifier success, and expected remediation time. Public vulnerability feeds can raise the priority of workloads that expose known exploited vulnerabilities.

The current scheduler consumes data/policy_metrics.json, which stores per-policy priors for success probability, expected latency, configured urgency flags, and baseline risk. The calibration pass (data/risk/policy_risk_map.json) augments those priors with observed detection/resolution counts, while data/risk/risk_calibration.csv captures corpus-level ΔR and residual risk (Table 10). Future iterations will enrich each queue item with container-image vulnerability joins (via Trivy/Grype), exploit-prior feeds [11], [13], and CISA KEV catalog checks [12] so that R reflects both exposure (Pod Security level, dangerous capabilities, host mounts) and exploit likelihood. The risk score R then feeds the bandit scoring function, allowing us to report absolute risk and per-patch risk reduction ΔR as first-class metrics.

4.8 Guidance Refresh Hooks

We curate policy guidance under docs/policy_guidance/raw/; scripts/refresh_guidance.py refreshes Pod Security, CIS, and Kyverno snippets (backed by docs/policy_guidance/sources.yaml) to keep guardrails current. LLM-backed proposer modes can retrieve these snippets at prompt time; future work will add richer metadata, cached verifier failures, and targeted passage retrieval when retries occur. This keeps the prompt budget bounded while grounding fixes in up-to-date hardening language without making retrieval a core evaluated contribution.

Table 12
Guardrail example: Cilium DaemonSet patch (excerpt).

Before	After
`securityContext:`	`securityContext:`
`privileged: true`	`privileged: false`
`allowPrivilegeEscalation: true`	`allowPrivilegeEscalation: false`
`capabilities:`	`capabilities:`
`add:`	`drop:`
`- NET_ADMIN`	`- ALL`
	`seccompProfile:`
	`type: RuntimeDefault`

Guardrails summarized in docs/privileged_daemonsets.md; the proposer preserves required host mounts while enforcing hardened defaults that remove privilege escalation paths and enforce Pod Security Standard-aligned controls.

Table 13
Managed-cloud cross-cluster replication results.

Cluster	Manifests	Success	Acceptance
EKS	200	198/200 (99.0%)	198/198 (100%)
GKE	200	200/200 (100%)	200/200 (100%)
AKS	200	197/200 (98.5%)	197/197 (100%)

Artifacts: per-cluster summaries and results under data/cross_cluster/, including eks/summary.csv, gke/summary.csv, and aks/summary.csv. **Collection:** replay steps in docs/cross_cluster_replay.md.

4.9 Risk-Bandit Scheduler with Aging and Urgency Preemption

Overview. The scheduler treats accepted remediation candidates as a cloud operations queue: high-risk items are served early, while aging prevents lower-risk work from remaining indefinitely unscheduled.

Notation. R_i denotes the risk units for item i; p_i is its empirical verifier success probability; $\mathbb{E}[t_i]$ is the observed proposer+verifier latency; wait_i tracks queue age; kev_i equals the configured KEV-style urgency boost when the item is marked for urgent handling (otherwise 0); and ε is a small positive floor preventing division by zero.

$$S_i = \frac{R_i \cdot p_i}{\max(\varepsilon, \mathbb{E}[t_i])} + \text{explore}_i + \alpha\,\text{wait}_i + \text{kev}_i \quad (1)$$

To make R_i auditable we now spell out its construction instead of burying it in the supplemental appendix. Each detection maps to a policy identifier; we pull the static weight w_{policy} from `data/risk/policy_risk_map.json`, add the configured KEV-style urgency surcharge κ when a violation is marked for urgent handling, and scale by the configured exploit-prior field e_{policy} captured in `data/policy_metrics.json`. Formally,

$$R_i = (w_{\text{policy}} + \kappa \cdot \mathbf{1}_{\text{urgent}}) \cdot e_{\text{policy}},$$

with $\kappa = 25$ risk units in the current configuration. p_i is the on-line verifier pass rate for that policy (accepted / attempted counts in `data/policy_metrics.json`), and $\mathbb{E}[t_i]$ is the running average of proposer+verifier latency recorded in the same file. We also report $\Delta R_i = R_i - R_i^{\text{post}}$ for every accepted patch, summing per corpus to produce Table 10. The supplemental appendix provides a numeric worked example using the same notation.

This scheduling function defines the score used today, where R_i is the risk score, p_i the empirical success rate, $\mathbb{E}[t_i]$ the observed latency, wait_i the queue age, and kev_i the configured KEV-style urgency boost, mirroring UCB-style bandit heuristics [23]. p_i and $\mathbb{E}[t_i]$ are refreshed from proposer/verifier telemetry; exploration uses an upper-confidence term and aging ensures fairness. The evaluation in Section 4.4 contrasts this bandit against FIFO, showing substantial reductions in top-risk wait time. Future work will incorporate additional risk signals and batch-aware policies, but the current heuristic already delivers measurable gains.

4.10 Baselines and Ablations

Replay of the 830-item queue snapshot (data/metrics_schedule_compare.json) quantifies how each scheduler treats critical detections. All heuristics clear roughly the same workload—$\Delta R/t = 247.2$ risk units per hour—because proposer/verifier throughput dominates. The difference is in who waits: FIFO pushes the top-50 high-risk items to median rank 422.5 (P95 620) and P95 wait 102.3 h, while the risk-only variant ($R/\mathbb{E}[t]$) and the full bandit (risk, aging, configured urgency boost, exploration) keep the same cohort within median rank 25.5 (P95 48) and cap top-risk P95 wait at 13.0 h. Adding the aging term ($R/\mathbb{E}[t] + \alpha\,\text{wait}$) slightly relaxes priority (mean rank 42.2, P95 124) but preserves the low top-risk wait (13.0 h) needed for fairness.

A finer-grained sweep over exploration and aging coefficients (data/metrics_schedule_sweep.json) shows the bandit sustaining high-risk median wait of 17.3 h (P95 32.8 h) even when exploration weight is set to 1.0, while low-risk items absorb most of the slack (median 120.9 h). The condensed simulation in data/operator_ab/summary_simulated.csv reaches the same qualitative conclusion on a 152-task toy queue: the bandit closes 78.9% of assignments with mean wait 0.23 h and P95 0.91 h, versus FIFO's 0.71 h mean and 1.69 h P95.

Table 14 quantifies how each verifier gate contributes to rejecting incomplete fixes. Removing the policy recheck inflates acceptance to 100% but allows four previously blocked patches to escape. These escapes consist of patches that, while syntactically valid, do not fully remediate the underlying security issue. For example, a patch might remove a privileged container but fail to drop the `SYS_ADMIN` capability, or it might set resource limits without also setting

Table 14
Verifier gate ablation using 19 patched samples (data/ablation/verifier_gate_metrics.json). Acceptance reports the share of patches passing under the scenario; escapes count regressions that the full verifier blocks.

Scenario	Disabled Gate(s)	Acceptance (%)	Escapes
Full	–	78.9	0
No-policy	policy	100.0	4
No-guardrail	no-new-violation	78.9	0
No-schema	kubectl	78.9	0
No-rescan	rescan	78.9	0

requests. The policy re-check gate catches these subtle regressions. The other gates leave acceptance unchanged at 78.9%. Figure 4 summarizes acceptance across rules-only, LLM-only, and hybrid modes. The Kyverno CLI baseline (scripts/run_kyverno_baseline.py, data/baselines/kyverno_baseline.csv) achieves 67.98% mean acceptance across 17 policies against the supported corpus; our verifier-guarded run reports 78.9% (+10.92 pp) while adding schema validation and dry-run checks. The gap between our CLI simulation (67.98%) and our mutating-webhook baseline on a fully seeded cluster (80–95% across overlapping policies) reflects missing production context (service accounts, host configuration) unavailable to offline CLI evaluation; Kyverno itself does not publish a global acceptance range.

Definition note. An "ablation" turns off a verifier check to see what breaks; an "escape" is a bad fix that would have slipped through without that check.

Case Study: A Patch Escape. The verifier's policy re-check gate is critical for preventing regressions. In one case, a patch was generated to address a 'hostPath' volume violation. The patch removed the 'hostPath' field but did not fully clear the asserted policy condition under the verifier's policy re-check. Without that gate, the patch would have been accepted despite incomplete remediation. The diff below shows the subtle but important change that the policy re-check gate caught.

```
--- a/manifest.yaml
+++ b/manifest.yaml
@@ -8,4 +8,4 @@
  volumes:
  - name: host-data
    hostPath:
-       path: /var/lib/data
+       emptyDir: {}
```

4.11 Metrics and Measurement

We formally define how we measure effectiveness and fairness:

Auto-fix Rate
$\frac{\#\text{ patches that pass the Verifier triad}}{\#\text{ detected violations}}$.

No-new-violations Rate
$\frac{\#\text{ accepted patches with zero new policy/schema violations}}{\#\text{ accepted patches}}$.

Patch Minimality
Median number of JSON Patch operations per accepted patch.

Time-to-patch
Wall-clock time from item enqueue to accepted patch; we report P50/P95 overall and for the top-risk decile.

Risk Reduction
For item i, $\Delta R_i = R_i^{\text{pre}} - R_i^{\text{post}}$. We report sum and rate: $\sum_i \Delta R_i$ and $\frac{\sum_i \Delta R_i}{\text{hour}}$.
Worked example. The supplemental appendix walks through a concrete queue item showing how we compute R, ΔR, and $\Delta R/t$ from the released telemetry.

Throughput
Accepted patches per hour.

Fairness
P95 wait time (broken out by risk tier) plus the starvation rate, defined as the fraction of items that wait more than 24 hours before scheduling. Both metrics are recomputed from the queue replays in data/scheduler/fairness_metrics.json.

Latest Evaluation. Running the checked full corpus of 15,718 detections in rules+guardrails mode yields 13,338 accepted out of 13,373 patched items (99.74%; auto-fix rate 0.8486 over detections) with a median of 9 JSON Patch operations. The unpatched remainder comes from detections we do not currently support or cases halted by guardrails before a patch is attempted; the 35 patched-but-unaccepted items failed policy/schema/no-new-violation/dry-run gates and were withheld. Bandit scheduling preserves fairness in deterministic queue replay: baseline top-risk items see P95 wait of 13.0 h at roughly 6.0 patches/hour while FIFO defers the same cohort to 102.3 h (+89.3 h).

Targets (Acceptance Criteria). Based on industry standards and research objectives, we target: Detection F1 ≥ 0.85 (hold-out), Auto-fix Rate $\geq 70\%$, No-new-violations Rate $\geq 95\%$, and median JSON Patch operations ≤ 9 (full corpus yields median 9 per data/metrics_latest.json).

5 Related Work and Baseline Positioning

Existing tools cover pieces of the cloud remediation workflow but rarely close the loop from detection to verified patch acceptance. GenKubeSec [17] uses LLM prompts to highlight issues and suggest fixes (we cite the authors' reported precision/recall and did not reproduce their pipeline). Kyverno [19] enforces policies at admission time yet does not rank work by risk or seed missing namespaces and CRDs. Large-scale SRE playbooks such as Borg [21] automate infrastructure repairs but are not aimed at hardening application manifests. Our approach combines deterministic repair rules with optional LLM backends behind the same verifier triad, runs on existing manifests rather than only admission requests, and orders accepted work by risk. Table 1 and Table 2 separate these qualitative differences from the reproduced head-to-head slice.

How we differ. We default to deterministic rules, add LLMs only behind the same guardrails, and accept a patch only after policy checks, schema checks, and a server-side dry-run pass. We run on existing manifests (not just admission-time) and schedule by risk instead of FIFO. The paper-facing metrics are regenerated from released data and scripts.

SAST vs. DAST Positioning. Many tools only scan manifests offline (SAST). Our verifier also exercises a live API server with `kubectl apply -dry-run=server` (DAST) so that cluster-specific problems—missing CRDs, namespaces, or quotas—are caught before apply. This mix of

Inputs: queue Q, risk R_i, urgency flag, wait time wait_i, bandit priors p_i, $\mathbb{E}[t_i]$, aging α, exploration coefficient β, urgency boost κ
while Q not empty **do**
  Score all items: For each $i \in Q$, compute $\text{base} = \frac{R_i \cdot p_i}{\max(\varepsilon, \mathbb{E}[t_i])}$; $\text{kev} = \kappa$ if urgency flag else 0; $\text{explore} = \beta\sqrt{\frac{\ln(1+n)}{1+n_i}}$; $S_i = \text{base} + \text{explore} + \alpha\,\text{wait}_i + \text{kev}$
  Pick $j = \arg\max_i S_i$; generate a JSON Patch for j using the selected proposer mode; run Verifier (policy, schema, server dry-run)
  if Verifier success **then**
    Apply patch; update counts (n_j, r_j) and online estimates p_j, $\mathbb{E}[t_j]$; remove j from Q
  else
    Update p_j, $\mathbb{E}[t_j]$ with failure; if retries< 3 then requeue j with feedback; otherwise drop j
  end if
  Age all items: $\text{wait}_i \leftarrow \text{wait}_i + \Delta t$
end while

Figure 6. Risk-Bandit scheduling loop (aging + urgency preemption) maximizing expected risk reduction per unit time with exploration and fairness.

static checks plus dry-run underpins the fixture-seeded live-cluster replay result (1,000/1,000 dry-run/apply acceptance).

Kubernetes has become the de facto operating substrate for cloud applications, yet its declarative configuration model remains prone to security misconfigurations [1], [2], [16], [17]. Admission controllers [3] such as Kyverno and OPA Gatekeeper can block insecure manifests, but blocking alone does not produce a reviewed patch for existing non-compliant resources. LLM-based remediation systems are promising [16], [17], but unguarded edits to security-critical infrastructure can introduce hallucinated fields, schema regressions, or incomplete fixes. We therefore position `k8s-auto-fix` as a post-hoc remediation system: detectors and policy engines still identify violations, but proposed patches are accepted only after independent verification.

1. Detection-Only Pipelines. Static analysis tools like `kube-linter` and policy engines such as Kyverno and OPA Gatekeeper excel at identifying misconfigurations ([5], [19], [4]). However, their core function is detection and admission control, not the generation of validated, minimal patches. Our work uses these powerful tools as the *Detector* and *Verifier* components in a broader remediation workflow.

2. Lack of Closed-Loop Verification. Few remediation pipelines enforce a rigorous, multi-gate verification process. A key novelty of our approach is the Verifier's triad of checks: a policy re-check to confirm the original violation is gone, schema validation to ensure correctness, and a server-side dry-run (`kubectl apply -dry-run=server`) to simulate the application of the patch against the Kubernetes API server, ensuring no new violations are introduced ([8]).

3. Inefficient Prioritization. Security work queues are often processed in a First-In, First-Out (FIFO) manner, so severe items can wait behind lower-risk work. Our scheduler assigns higher priority to items with larger risk reduction and higher empirical completion probability while adding age-based priority so lower-risk items continue to make progress.

Emerging Agents. Concurrent with this work, OpenAI introduced *Aardvark*, later renamed *Codex Security* in a March 2026 research preview [25], as an AI security agent for automated vulnerability finding and patching. Similarly, *KubeIntellect* [26] proposes an LLM-orchestrated agent for general Kubernetes management. While these agents target general software vulnerabilities or broad cluster operations, our work specifically addresses the Kubernetes configuration domain, enforcing domain-specific invariants (schema, policy, dry-run) that general-purpose code agents may miss. Furthermore, we provide a fully open-source, reproducible pipeline focused on Kubernetes manifest remediation.

6 Limitations and Mitigations

The prototype prioritizes shipping guardrails and evidence, but several constraints remain before production deployment. **External validity** is limited because the supported and optional-LLM corpora skew toward Helm-derived workloads and may miss bespoke production clusters; we refresh the ArtifactHub scrape monthly (`scripts/collect_artifacthub.py`), add partner manifests as they are shared, and run an 8–12 analyst rotation with the survey instrument in docs/operator_survey.md so live results supplement deterministic replays. **Fixture sensitivity** means verifier success depends on seeding CRDs, namespaces, and service accounts that mirror production; the fixture harness (infra/fixtures/) now auto-installs required objects before replay, and the pending dynamic discovery prototype records missing fixtures at runtime so we can ship cluster-specific bundles with the artifact release. **LLM latency gaps** persist because API-backed calls add seconds relative to rules mode; we cache prompt templates, stream telemetry to data/grok5k_telemetry.json, fall back to deterministic rules when wall-clock thresholds are exceeded, and are validating smaller hosted models behind the same guardrails. **Deterministic scheduler replays** mean reported fairness metrics come from queue replays rather than live handoffs; we publish the replay traces (data/outputs/scheduler/) and will pair them with the logged human-in-the-loop rotation so reviewers can compare deterministic and live outcomes once the study completes.

7 Conclusion and Future Work

The current pipeline records 1,000/1,000 fixture-seeded live-cluster dry-run/live-apply acceptance with zero rollbacks (Table 11, data/live_cluster/results_1k.json). Across offline

corpora, the system delivers 93.54% acceptance on the 5k supported corpus, 100.00% on the 1,264-manifest supported slice, 100.00% on the 1,313-manifest API-backed LLM run, and 88.52% on the optional LLM 5k sweep overall, while deterministic rules + guardrails accept 13,338 / 13,373 patched items (99.74%; auto-fix rate 0.8486 over 15,718 detections) with median patch ops 9 (Table 11, data/metrics_latest.json). The risk-aware scheduler trims top-risk P95 wait times from 102.3 h (FIFO) to 13.0 h in deterministic queue replay (data/scheduler/metrics_sweep_live.json, data/outputs/scheduler/metrics_schedule_sweep.json).

The paper-facing summary tables are regenerated from the public artifact bundle (`make reproducible-report`, ARTIFACTS.md), and the scheduler comparisons we report stem from deterministic queue replays rather than live analyst rotations. These gains are anchored in deterministic guardrails, schema validation, and server-side dry-run enforcement, with matching API-backed LLM runs available to practitioners who can supply provider credentials and budget roughly $1.22 per 5k sweep under the published pricing (data/grok5k_telemetry.json, [9]). To prevent configuration drift, every accepted patch is surfaced as a pull request through our GitOps helper (scripts/gitops_writeback.py), which records verifier evidence, captures the JSON Patch diff, and requires human approval before merge, mirroring the workflow detailed in docs/GITOPS.md.

Looking forward, we will automate guidance refreshes in CI (scripts/refresh_guidance.py), fold richer exploit-prior feeds into the risk score R_i, and scale the qualitative feedback loop that now captures operator notes in docs/qualitative_feedback.md. As the LLM-backed proposer matures, we plan to publish comparative acceptance and latency data, extend scheduler policies with batch-aware fairness, and run human-in-the-loop rotations so the prototype can mature into a broader remediation workflow.

Near-term efforts focus on keeping the seeded fixtures current so the 1,000/1,000 live-cluster outcome persists for new corpora, broadening Kyverno webhook baselines across additional policy families and alternative clusters, enriching optional-LLM telemetry with monotonic latency traces, and conducting an operator rotation with embedded surveys to validate the scheduler against real analyst workflows. All artifacts remain available at https://github.com/bmendonca3/k8s-auto-fix.

References

[1] CIS Kubernetes Benchmarks. Accessed: Jun. 2026. [Online]. Available: https://www.cisecurity.org/benchmark/kubernetes

[2] Kubernetes: Pod Security Standards. Accessed: Jun. 2026. [Online]. Available: https://kubernetes.io/docs/concepts/security/pod-security-standards/

[3] Kubernetes: Admission Controllers. Accessed: Jun. 2026. [Online]. Available: https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/

[4] OPA Gatekeeper How-to. Accessed: Jun. 2026. [Online]. Available: https://open-policy-agent.github.io/gatekeeper/website/docs/howto/

[5] *kube-linter* Documentation. Accessed: Jun. 2026. [Online]. Available: https://docs.kubelinter.io/

[6] Kubernetes: Configure a Security Context. Accessed: Jun. 2026. [Online]. Available: https://kubernetes.io/docs/tasks/configure-pod-container/security-context/

[7] RFC 6902: JSON Patch, DOI:10.17487/RFC6902. Accessed: Jun. 2026. [Online]. Available: https://www.rfc-editor.org/info/rfc6902

[8] `kubectl apply` Command Reference (server-side dry-run). Accessed: Jun. 2026. [Online]. Available: https://kubernetes.io/docs/reference/kubectl/generated/kubectl_apply/

[9] xAI, "Reasoning API Pricing." Accessed: Jun. 2026. [Online]. Available: https://console.x.ai/pricing

[10] Kubernetes: Seccomp and Kubernetes. Accessed: Jun. 2026. [Online]. Available: https://kubernetes.io/docs/reference/node/seccomp/

[11] NIST National Vulnerability Database (NVD). Accessed: Jun. 2026. [Online]. Available: https://nvd.nist.gov/

[12] CISA Known Exploited Vulnerabilities Catalog. Accessed: Jun. 2026. [Online]. Available: https://www.cisa.gov/known-exploited-vulnerabilities-catalog

[13] FIRST Exploit Prediction Scoring System (EPSS). Accessed: Jun. 2026. [Online]. Available: https://www.first.org/epss/

[14] Trivy: Vulnerability Scanner for Containers and IaC. Accessed: Jun. 2026. [Online]. Available: https://aquasecurity.github.io/trivy/

[15] Grype: A Vulnerability Scanner for Container Images and Filesystems. Accessed: Jun. 2026. [Online]. Available: https://github.com/anchore/grype

[16] Z. Ye, T. H. M. Le, and M. A. Babar, "LLMSecConfig: An LLM-Based Approach for Fixing Software Container Misconfigurations," in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, 2025.

[17] E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, "GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection, Localization, Reasoning, and Remediation," *arXiv preprint arXiv:2405.19954*, 2024.

[18] M. De Jesus, P. Sylvester, W. Clifford, A. Perez, and P. Lama, "LLM-Based Multi-Agent Framework For Troubleshooting Distributed Systems," in *Proc. of the 2025 IEEE Cloud Summit*, 2025 (author's version).

[19] Kyverno Project, "Kyverno Documentation," Accessed: Jun. 2026. [Online]. Available: https://kyverno.io/docs/

[20] Fairwinds, "Polaris Documentation." Accessed: Jun. 2026. [Online]. Available: https://polaris.docs.fairwinds.com/

[21] A. Verma *et al.*, "Large-scale Cluster Management at Google with Borg," in *Proc. EuroSys*, 2015; supplemental SRE updates accessed Jun. 2026. [Online]. Available: https://research.google/pubs/pub43438/

[22] ArtifactHub Documentation. Accessed: Jun. 2026. [Online]. Available: https://artifacthub.io/docs/

[23] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time Analysis of the Multiarmed Bandit Problem," *Machine Learning*, vol. 47, no. 2, pp. 235–256, 2002.

[24] M. Joseph, M. Kearns, J. H. Morgenstern, and A. Roth, "Fairness in Learning: Classic and Contextual Bandits," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[25] OpenAI, "Codex Security: Now in Research Preview," Mar. 2026. Accessed: Jun. 2026. [Online]. Available: https://openai.com/index/codex-security-now-in-research-preview/

[26] M. S. Ardebili and A. Bartolini, "KubeIntellect: A Modular LLM-Orchestrated Agent Framework for End-to-End Kubernetes Management," *arXiv preprint arXiv:2509.02449*, 2025.

Brian Mendonca has worked as an Aerospace Quality Engineer at BAE Systems (2024–2025) and at Tube Specialties Inc. (2025–present), where he led Lean/Six Sigma continuous improvement, nonconformance management, and 8D root-cause investigations supporting compliance and on-time delivery.

He received the B.E. in Mechanical Engineering (summa cum laude, GPA 3.99) from Arizona State University in 2021 and is currently pursuing his MS degree in Computer Science at Georgia Tech. His research interests include secure configuration management for cloud-native systems, program analysis for infrastructure-as-code, and data-informed quality engineering.

Vijay K. Madisetti holds the position of Professor of Cybersecurity and Privacy (SCP) at Georgia Tech. He earned his PhD in Electrical Engineering and Computer Sciences from the University of California at Berkeley and is recognized as a Fellow of the IEEE. Additionally, he has been honored with the Terman Medal by the American Society of Engineering Education (ASEE), and was awarded Georgia Tech's Outstanding PhD Dissertation Advisor Award. Professor Madisetti has authored several widely referenced textbooks on topics including cloud computing, data analytics, blockchain, and microservices. Dr. Madisetti is co-author of the widely used IEEE VHDL RTL Synthesis Language Standard, and inventor on over 80 issued US patents.